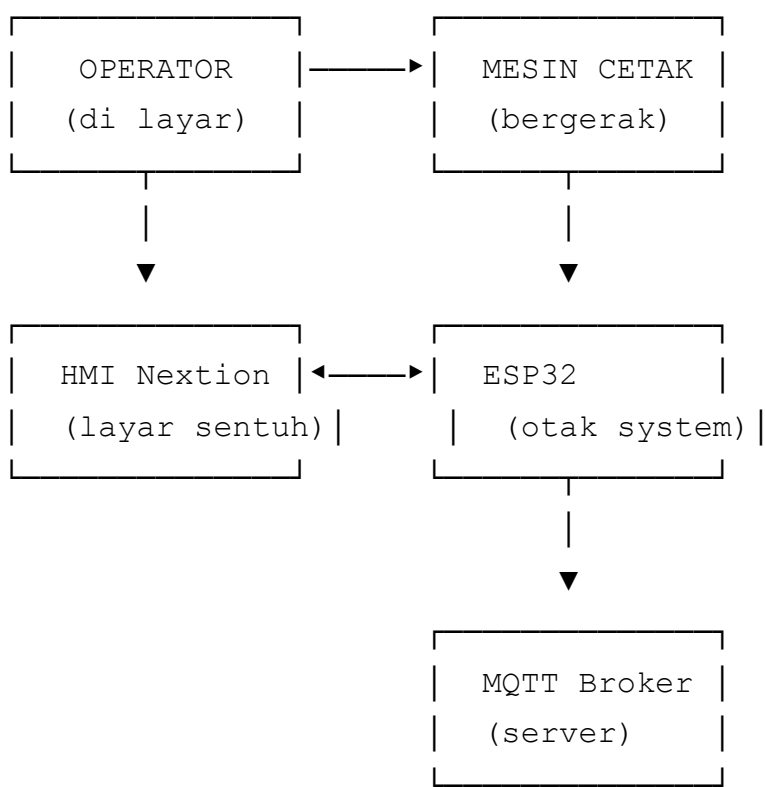


CaesarFirmwareV1

Firmware ESP32 yang mengontrol mesin cetak Caesar. Semua operasi – login operator, input mould, input lot, pencacahan siklus, pencatatan waktu henti, pencatatan produk cacat, hingga pengiriman data ke server – berjalan dari firmware ini.

Cara Kerja



Simulator kirim sinyal START. Firmware hitung 5 gerakan naik-turun. Simulator kirim sinyal siklus selesai. Firmware tambah jumlah siklus, kurangi kuota, update layar, kirim data ke server.

Fitur

Bagian	Fungsi
Login	Operator login sebelum jalankan mesin
Mould	Input cetakan (model, jumlah lubang)
Lot	Input nomor lot dan target produksi
Siklus	Hitung berapa kali mesin mencetak

Bagian	Fungsi
Downtime	Catat waktu mesin berhenti dan alasannya
NG	Catat produk cacat
Interlock	Pengaman: mesin tidak siap, siklus tidak jalan
Waktu	Jam WIB dari internet (NTP)
Simulasi	Mode otomatis untuk testing

File

config.h

Pengaturan utama. Semua bisa diubah tanpa menyentuh kode program.

WiFi:

- SSID: `TIMEO`
- Password: `1234Saja`

MQTT:

- Broker: `broker.hivemq.com`
- Port: `1883`
- Device ID: `91c21b8614cd`

Topik MQTT:

Topik	Fungsi
<code>finish/IOTHP-BP/91c21b8614cd</code>	Menerima sinyal siklus selesai dari simulator
<code>dataA/IOTHP-BP/91c21b8614cd</code>	Mengirim data layer depan (retained)
<code>dataB/IOTHP-BP/91c21b8614cd</code>	Mengirim data layer belakang (retained)
<code>event/IOTHP-BP/91c21b8614cd</code>	Mengirim event downtime dan NG
<code>CONTROL/IOTHP-BP/91c21b8614cd</code>	Menerima perintah REBOOT

Waktu:

- Server NTP: `pool.ntp.org`
- Zona waktu: WIB (UTC+7)

Tabel Data:

Tabel	Isi
Operator	ID 1111-3333, nama lengkap
Mould	Kode 1111-3333, nama model, jumlah cavity
Lot	Kode 1111-3333, nama model, target produksi

Edit tabel di `config.h` sesuai data produksi aktual.

CaesarFirmwareV1.ino

File utama yang menghubungkan semua bagian. Berisi deklarasi komponen HMI, variabel global, dan alur utama program.

Variabel global:

- `WiFiClient wifiClient` – objek koneksi WiFi
- `PubSubClient mqttClient` – objek koneksi MQTT, pakai `wifiClient` sebagai transport
- `char cycleStartTime[32] / char cycleFinishTime[32]` – menyimpan waktu mulai dan selesai siklus terakhir dari simulator
- `LayerComponents` – struct berisi nama-nama komponen di HMI untuk tiap layer (cycle, output, ok, ng, quota, isi, target, model, lot, operatorName, operatorId)
- `LayerState` – struct berisi nilai produksi aktual untuk satu layer
- `uint8_t currentPageId` – nomor halaman HMI yang sedang aktif. Scanner membaca nilai ini untuk tahu harus kirim input ke mana.

Deklarasi komponen HMI:

Semua halaman (`NexPage`), tombol (`NexButton`), teks (`NexText`), angka (`NexNumber`), dan variabel (`NexVariable`) dideklarasikan di sini. Setiap komponen punya nomor halaman, nomor komponen, dan nama. Contoh: `NexButton bOkF = NexButton(1, 2, "bOkF")` artinya tombol "bOkF" di halaman 1, komponen nomor 2.

`nex_listen_list` – array semua komponen yang harus didengarkan.

`nexLoop(nex_listen_list)` di `loop()` akan memeriksa apakah ada tombol yang ditekan operator.

`setup()` – dijalankan sekali saat ESP32 dinyalakan:

1. `Serial.begin(115200)` – nyalakan serial monitor untuk debug

2. `SCANNER_SERIAL.begin(...)` — nyalakan Serial2 untuk barcode scanner (RX=GPIO16, TX=GPIO17, 9600 baud)
3. `initTimeSource()` — set zona waktu WIB atau inisialisasi modul RTC
4. `initInterlock()` — set GPIO25 sebagai OUTPUT, matikan dulu (LOW)
5. `registerLoginCallbacks()` — daftarkan semua callback tombol login, logout, dan navigasi halaman
6. `registerInputCallbacks()` — daftarkan callback tombol mould, lot, dan NG submit
7. `registerDowntimeCallbacks()` — daftarkan callback tombol downtime (mulai dan clear)
8. `registerSimulationCallbacks()` — daftarkan callback tombol auto-cycle
9. `setupWifi()` — mulai koneksi WiFi
10. `setupMqtt()` — set broker dan callback MQTT

`loop()` — dijalankan ratusan kali per detik:

1. `nexLoop(nex_listen_list)` — baca semua tombol di HMI. Jika ada yang ditekan, panggil callback yang terdaftar.
2. `mqttLoop()` — proses koneksi WiFi, MQTT, dan pesan masuk
3. `syncTimeSource()` — ambil waktu dari NTP jika belum sinkron
4. `handleScanner()` — baca input dari barcode scanner
5. `updateTnow()` — kirim timestamp lengkap ke `pageSys.tNow.txt` tiap 1 detik
6. `updateSimulation()` — jalankan auto-cycle jika mode simulasi aktif

wifi_mqtt.ino

Mengatur koneksi WiFi dan MQTT. Semua komunikasi dengan server melewati file ini.

Variabel global:

- `lastWifiAttempt` — timestamp kapan terakhir coba sambung WiFi. Digunakan untuk delay 5 detik agar ESP32 tidak spam reconnect.
- `lastMqttAttempt` — timestamp kapan terakhir coba sambung MQTT. Delay sama, 5 detik.

`publishEvent(json)` — kirim event ke server (downtime/NG). Cek MQTT connected dulu. Publish non-retained ke `EVENT_TOPIC`. Non-retained artinya pesan tidak disimpan server — jika subscriber offline saat pesan dikirim, subscriber tidak akan terima pesan ini nanti.

`mqttCallback(topic, payload, length)` — handler saat terima pesan dari server. Dua jenis pesan yang dihandle:

1. Dari `CONTROL_TOPIC`, isi `REBOOT` (6 karakter) → restart ESP32 via `ESP.restart()`

2. Dari `FINISH_TOPIC` , isi JSON `1cycle` → parse JSON, ambil `startTime` dan `finishTime` , panggil `handleMqttCycle()`

Jika pesan bukan dari kedua topik ini → diabaikan.

`setupWifi()` — set WiFi mode sebagai station (bukan access point), set hostname ke `MQTT_CLIENT_ID` agar mudah dikenali di jaringan, mulai koneksi ke SSID dan password yang ditentukan.

`setupMqtt()` — set alamat broker dan port, daftarkan `mqttCallback` untuk pesan masuk, set buffer 256 byte (cukup untuk payload JSON siklus), set keep alive 60 detik (jika tidak ada aktivitas MQTT selama 60 detik, koneksi putus otomatis), set socket timeout 5 detik.

`reconnectWifi()` — jika WiFi sudah terhubung → skip. Jika belum dan sudah lewat 5 detik sejak percobaan terakhir → putuskan WiFi lalu sambung ulang. Delay 5 detik mencegah ESP32 mencoba sambung ulang terlalu sering.

`reconnectMqtt()` — logika sama. Jika MQTT sudah terhubung → skip. Jika belum dan sudah lewat 5 detik → coba `mqttClient.connect()` . Setelah terhubung → subscribe `CONTROL_TOPIC` (untuk REBOOT) dan `FINISH_TOPIC` (untuk siklus dari simulator).

`mqttLoop()` — fungsi utama yang dipanggil dari `loop()` . Urutan:

1. `reconnectWifi()` — coba sambung WiFi jika perlu
2. Set LED WiFi: HIGH jika terhubung, LOW jika putus
3. Jika WiFi belum terhubung → skip (tidak bisa proses MQTT tanpa WiFi)
4. `reconnectMqtt()` — coba sambung MQTT jika perlu
5. Jika MQTT terhubung → `mqttLoop()` untuk proses pesan masuk

cycle.ino

Penghitung siklus dari simulator. Dipanggil saat firmware terima JSON `1cycle` dari `FINISH_TOPIC` .

Variabel state:

- `frontCycle` / `backCycle` — jumlah siklus untuk layer depan dan belakang
- `frontOutput` / `backOutput` — jumlah output untuk tiap layer
- `frontQuota` / `backQuota` — sisa kuota produksi
- `frontIsi` / `backIsi` — isi per cetakan (dari mould)
- `frontNg` / `backNg` — jumlah produk cacat

`handleMqttCycle(startTime, finishTime)` — fungsi utama. Alur:

1. Baca `pageSys.nMReady` dari HMI. Jika `0` → interlock merah, siklus tidak jalan. Return.
2. Baca 11 nilai dari HMI: siklus, output, kuota, isi, NG untuk depan dan belakang. Jika gagal baca → return.
3. Cek kondisi: jika `frontIsi==0` atau `backIsi==0` → mould belum diisi. Jika `frontQuota < frontIsi` atau `backQuota < backIsi` → kuota habis. Dalam kedua kasus → update status saja, tidak increment siklus.
4. Simpan `startTime` dan `finishTime` ke variabel global.
5. Increment siklus: `frontCycle++`, `backCycle++`
6. Tambah output: `frontOutput += frontIsi`, `backOutput += backIsi`
7. Kurangi kuota: `frontQuota -= frontIsi`, `backQuota -= backIsi`
8. Hitung OK: `frontOk = frontOutput - frontNg`, `backOk = backOutput - backNg`
9. Update semua komponen HMI: siklus, output, kuota, OK untuk depan dan belakang
10. Update tampilan simulation dan dashboard
11. Update dashboard NG display
12. `updateDashboardStatus()` – hitung ulang status mesin (mungkin berubah karena kuota berkurang)
13. `publishBothLayers()` – kirim data lengkap ke server (retained)

Siklus selalu increment kedua layer sekaligus. Firmware tidak membedakan layer depan atau belakang dalam siklus.

dashboard.ino

Logika status mesin. Menentukan apakah mesin siap atau terkunci berdasarkan kondisi saat ini.

`getFrontDashboardStatus(login, machineDowntime, downtime, quota, cavity, isi)` – cek kondisi secara berurutan (prioritas dari atas):

1. `login == 0` → "BP DEPAN LOCKED" (operator belum login)
2. `machineDowntime != 0` → "BP DEPAN MC DOWNTIME" (mesin sedang downtime)
3. `downtime != 0` → "BP DEPAN DOWNTIME" (layer ini sedang downtime)
4. `quota < isi` → "BP DEPAN NO LOT" (kuota kurang dari isi per cetakan)
5. `cavity == 0` → "BP DEPAN NO MOULD" (belum pilih mould)
6. `isi == 0` → "BP DEPAN ISI 0" (isi per cetakan kosong)
7. Sisanya → "BP DEPAN READY"

`getBackDashboardStatus()` – logika sama untuk layer belakang.

`updateDashboardStatus()` – fungsi utama yang dipanggil dari banyak tempat:

1. Baca 11 nilai dari HMI: login depan/belakang, downtime mesin, downtime depan/belakang, kuota depan/belakang, cavity depan/belakang, isi depan/belakang
2. Jika gagal baca → set interlock LOW (fail-safe), return
3. Hitung status untuk depan dan belakang
4. Cek apakah keduanya READY → `machineReady`
5. Kirim status teks ke dashboard: `tFStat` , `tBStat`
6. Set `nFReady` , `nBReady` , `nMReady` di HMI
7. Set warna `tILock` : hijau(2024) jika siap, merah(63488) jika tidak
8. Kosongkan teks `tILock`
9. `setInterlock(machineReady)` — drive GPIO25 sesuai kondisi

`syncSimulationTargets()` — baca target dari `pageSys.nFTarget` / `nBTarget` , kirim ke `pageSim.nSTgtF` / `nSTgtB` . Dipanggil saat operator buka halaman simulasi.

downtime.ino

Pencatatan waktu henti mesin. Mengelola mulai dan clear downtime untuk layer depan, belakang, dan mesin.

`sendDowntimeText(format, value)` — helper untuk kirim perintah format ke HMI. Contoh:
`sendDowntimeText("pageSys.tFDtType.txt=\"%s\"", reason)` → kirim
`pageSys.tFDtType.txt="BAHAN TELAT"` .

`showDowntimeInfo(pageName)` — navigasi ke halaman info downtime:

1. `sendCommand("page pageDashboard")` — buka dashboard dulu
2. `delay(200)` — tunggu 200ms agar dashboard selesai render
3. `lockInterlock()` — kunci interlock
4. `delay(100)` — tunggu 100ms agar perintah interlock sampai
5. `sendDowntimeText("page %s", pageName)` — buka halaman info downtime

Dashboard harus muncul dulu karena komponen `tILock` ada di halaman dashboard. Jika langsung kirim perintah interlock saat masih di halaman downtime, perintah tidak sampai.

`lockInterlock()` — kunci mesin:

1. `setInterlock(false)` — GPIO25 LOW
2. `sendCommand("pageSys.nMReady.val=0")` — set mesin tidak siap
3. `sendCommand("tILock.bco=63488")` — warna interlock merah
4. `sendCommand("tILock.txt=\"%s\")` — kosongkan teks interlock

startFrontDowntime (reason) – mulai downtime layer depan:

1. Set `pageSys.nFDtAct.val=1` (flag downtime aktif)
2. Tulis tipe downtime ke `pageSys.tFDtType`
3. Copy waktu dari `pageSys.tNow.txt` ke `pageSys.tFDtStart.txt`
4. `lockInterlock()` – kunci mesin
5. Ambil timestamp dari NTP, buat JSON event, publish ke server
6. `updateDashboardStatus()` – update status (sekarang DOWNTIME)
7. `showDowntimeInfo("pageDtInfoF")` – buka halaman info downtime depan

startBackDowntime() / **startMachineDowntime()** – logika sama untuk layer belakang dan mesin.

clearFrontDowntime() – clear downtime depan:

1. Reset flag: `nFDtAct=0` , kosongkan tipe dan waktu
2. Set `pageSys.nDtPop.val=0`
3. Ambil timestamp, publish event `downtime_clear`
4. `updateDashboardStatus()` – status mungkin berubah ke READY
5. `sendCommand("page pageDashboard")` – kembali ke dashboard

clearBackDowntime() / **clearMachineDowntime()** – logika sama.

Callback tombol:

- `bBhnFCallback` → `startFrontDowntime("BAHAN TELAT")`
- `bTggPnsFCallback` → `startFrontDowntime("TUNGGU PANAS")`
- `bMoffFCallback` → `startFrontDowntime("MOULD OFF")`
- `bLbhPnsFCallback` → `startFrontDowntime("LEBIH PANAS")`
- `bKrgPnsFCallback` → `startFrontDowntime("KURANG PANAS")`
- `bTrialFCallback` → `startFrontDowntime("TRIAL")`
- `bRunFCallback` → `clearFrontDowntime()`
- `bBackDtFCallback` → kembali ke dashboard
- Back dan Machine punya callback setara

registerDowntimeCallbacks() – daftarkan semua callback ke tombol yang sesuai.

input.ino

Penerimaan input dari layar HMI dan barcode scanner. File terbesar (469 baris). Berisi semua handler untuk mould, lot, NG, dan helper functions untuk komunikasi dengan HMI.

Komunikasi dasar dengan HMI (Nextion):

sendInputText(component, value) – kirim teks ke komponen HMI. Format:

`{component}.txt="{value}"`. Contoh: `sendInputText("pageSys.tFMld", "1111")` →

kirim `pageSys.tFMld.txt="1111"`.

sendInputValue(component, value) – kirim angka ke komponen HMI. Format:

`{component}.val={value}`. Contoh: `sendInputValue("pageSys.nFCav", 8)` → kirim

`pageSys.nFCav.val=8`.

readNextionText(component, buffer, bufferSize) – baca teks dari HMI. Kirim perintah

`get {component}.txt`. Tunggu jawaban dari HMI (timeout 200ms). Header jawaban: 0x70. Data:

karakter biasa. Terminator: 3x 0xFF berturut-turut. Return true jika berhasil.

readNextionValue(component, value) – baca angka dari HMI. Kirim perintah `get`

`{component}.val`. Header jawaban: 0x71. Data: 4 byte little-endian (nilai angka). Terminator: 3x

0xFF. Return true jika berhasil.

Proses mould:

fillFrontMouldInput(code) – isi kolom input mould depan dengan kode barcode.

handleFrontMould() – proses input mould depan:

1. Baca kode dari `tInMF` (kolom input)
2. Cari di tabel `MOULDS` di `config.h`
3. Jika tidak ditemukan → tampilkan "MOULD NG", set cavity=0, isi=0
4. Jika ditemukan → tampilkan nama model, set cavity dan isi (jumlah lubang), update semua komponen HMI terkait (`pageSys`, `pageLot`, `pageDashboard`, `pageSim`), update dashboard status, kirim data ke server

clearFrontMould() – kosongkan semua field mould depan, reset cavity dan isi ke 0.

handleBackMould() / **clearBackMould()** – logika sama untuk layer belakang.

Proses lot:

fillFrontLotInput(code) – isi kolom input lot depan dengan kode barcode. Juga baca cavity dari HMI dan tampilkan di `nIsiLotF`.

handleFrontLot() – proses input lot depan:

1. Baca kode dari `tInF`
2. Cari di tabel `LOTS` di `config.h`

3. Jika tidak ditemukan → tampilkan "LOT NG"
4. Baca cavity dan isi dari HMI. Validasi: cavity harus >0 , isi harus >0 , isi $\leq$ cavity
5. Jika semua valid → set target dan kuota, update semua komponen HMI, update dashboard, kirim data ke server

`clearFrontLot()` – kosongkan semua field lot depan, reset target dan kuota ke 0.

`handleBackLot()` / `clearBackLot()` – logika sama untuk layer belakang.

Proses NG:

`publishNgEvent(layer, typeComponent, countComponent, acceptedComponent)` – kirim event produk cacat:

1. Baca `acceptedComponent` (flag dari HMI bahwa operator sudah submit). Jika 0 → return.
2. Baca `countComponent` (jumlah NG). Jika 0 → return.
3. Baca `typeComponent` (tipe NG, misal "SCRATCH"). Jika kosong → return.
4. Ambil timestamp dari NTP
5. Buat JSON:

```
{"event":"ng_submit","layer":"...","ng_type":"...","ng_count":...,"timestamp":"..."}
```

6. Publish ke server
7. Clear jumlah NG dan flag submit di HMI

`bOkNgFCallback()` → `publishNgEvent("FRONT", "pageSys.tFNgType", "nNgInF", "pageSys.nFNgEvent")`

`bOkNgBCallback()` → `publishNgEvent("BACK", "pageSys.tBNgType", "nNgInB", "pageSys.nBNgEvent")`

`registerInputCallbacks()` – daftarkan semua callback tombol mould, lot, dan NG.

interlock.ino

Pengaman mesin via GPIO25. File kecil (10 baris) dengan dua fungsi.

`initInterlock()` – panggilan sekali saat `setup()` :

1. `pinMode(INTERLOCK_PIN, OUTPUT)` – set GPIO25 sebagai output
2. `digitalWrite(INTERLOCK_PIN, LOW)` – matikan dulu (mesin terkunci saat baru nyala)

`setInterlock(ready)` – set kondisi interlock:

- `ready = true` → `digitalWrite(INTERLOCK_PIN, HIGH)` → mesin siap

- `ready = false` → `digitalWrite(INTERLOCK_PIN, LOW)` → mesin terkunci

Dipanggil dari dua tempat:

1. `updateDashboardStatus()` — berdasarkan kalkulasi kondisi mesin (login, downtime, mould, lot, isi, kuota)
 2. `lockInterlock()` di `downtime.ino` → langsung set LOW saat downtime dimulai (tanpa menunggu kalkulasi)
-

login.ino

Login dan logout operator. Mengelola autentikasi operator sebelum mesin bisa dijalankan.

`sendFormattedCommand(format, value)` — helper untuk kirim perintah format ke HMI. Dua versi: untuk string dan untuk angka.

`setFrontLogin(id, name)` — set login depan:

1. `pageSys.frontLogin.val=1` — flag login aktif
2. `pageSys.nFOpId.val={id}` — simpan ID operator
3. `pageSys.tFOpName.txt="{name}"` — simpan nama operator
4. Update kolom nama di halaman login
5. Update dashboard: tampilkan nama operator
6. `updateDashboardStatus()` — status mungkin berubah ke READY
7. `publishBothLayers()` — kirim data ke server (termasuk nama operator)

`setBackLogin()` — logika sama untuk layer belakang.

`clearFrontLogin()` — logout depan:

1. Set `currentPageId = 0` (kembali ke dashboard)
2. Reset semua field: login=0, ID=0, nama kosong
3. Update kolom nama di dashboard: "Silahkan Login"
4. `updateDashboardStatus()` — status berubah ke LOCKED
5. `publishBothLayers()` — kirim data ke server
6. Kembali ke halaman dashboard

`clearBackLogin()` — logika sama.

`handleFrontLogin()` — proses login depan:

1. Baca ID dari `nIdF` (kolom input angka)

2. Cari di tabel `OPERATORS` di `config.h`
3. Jika tidak ditemukan → tampilkan "ID SALAH"
4. Jika ditemukan → panggil `setFrontLogin(id, name)`

`handleBackLogin()` – logika sama untuk belakang.

Callback tombol:

- `bOkFCallback` → `handleFrontLogin()`
- `bOkBCallback` → `handleBackLogin()`
- `bLogoutFCallback` → `clearFrontLogin()`
- `bLogoutBCallback` → `clearBackLogin()`
- `bBackFCallback` / `bBackBCallback` → `set currentPageId = 0`

`pageCallback(ptr)` – dipanggil saat operator pindah halaman. Simpan `currentPageId` dari pointer. Jika masuk halaman simulasi → `syncSimulationTargets()` .

`registerLoginCallbacks()` – daftarkan callback untuk semua halaman (`pageDashboard`, `pageLoginF`, `pageLoginB`, dll) dan semua tombol login/logout/back.

publish.ino

Pengiriman data kondisi mesin ke server. Data dikirim dalam format JSON dan disimpan di server (retained).

`FRONT_COMPONENTS` / `BACK_COMPONENTS` – konstanta berisi nama-nama komponen HMI untuk tiap layer. Contoh: `"pageSys.nFCyc"` untuk siklus depan, `"pageSys.tFModel"` untuk model depan.

`readLayerState(components, state)` – baca semua nilai dari HMI untuk satu layer:

1. `memset(state, 0, sizeof(*state))` – kosongkan struct
2. Baca 8 nilai angka: cycle, output, ok, ng, quota, isi, target, operatorId
3. Baca 3 nilai teks: model, lot, operatorName
4. Return true jika semua berhasil dibaca

`publishLayerState(topic, components)` – publish data satu layer ke server:

1. Cek MQTT connected
2. `readLayerState()` – baca semua nilai dari HMI
3. Buat `JsonDocument` (ArduinoJson v7)

4. Isi semua field: cycle, output, ok, ng, quota, isi, target, model, lot, operator, operator_id, startTime, finishTime
5. Serialize ke buffer 384 byte
6. `mqttClient.publish(topic, payload, true)` — publish retained

Retained artinya server menyimpan pesan terakhir. Subscriber yang baru connect akan langsung terima data terakhir tanpa menunggu publish baru.

`publishBothLayers()` — panggil `publishLayerState()` untuk depan (`DATA_A_TOPIC`) dan belakang (`DATA_B_TOPIC`).

rtc.ino

Sumber waktu. Mendukung dua mode: NTP (internet) dan RTC (jam hardware). Hanya satu mode aktif, diatur di `config.h`.

Variabel NTP:

- `ntpConfigured` — apakah sudah pernah jalankan `configTzTime()` (cukup sekali)
- `ntpReady` — apakah waktu dari NTP sudah berhasil diambil
- `lastNtpAttempt` — kapan terakhir coba ambil waktu
- `NTP_RETRY_INTERVAL` — interval retry 1 detik

Variabel RTC:

- `rtc` — objek `RTC_DS3231` untuk komunikasi I2C
- `rtcReady` — apakah modul RTC terdeteksi

Variabel umum:

- `lastRtcUpdate` — kapan terakhir update `tNow` di HMI
- `RTC_UPDATE_INTERVAL` — interval update 1 detik

`sendRtcText(value)` — kirim timestamp ke `pageSys.tNow.txt` di HMI.

`initTimeSource()` — inisialisasi sumber waktu:

- **NTP mode:** `setenv("TZ", WIB_TIMEZONE, 1) + tzset()` → set zona waktu WIB. Tidak perlu internet.
- **RTC mode:** `Wire.begin()` → mulai I2C. `rtc.begin()` → deteksi modul. Jika `rtc.lostPower()` → set waktu dari compiled time. Set `rtcReady = true`.

`syncTimeSource()` — sinkronisasi waktu:

- **NTP mode:** skip jika sudah ready, WiFi belum nyambung, atau belum lewat 1 detik. Jalankan `configTzTime()` sekali pertama. Coba `getLocalTime()`. Jika berhasil → `ntpReady = true`.
- **RTC mode:** tidak perlu sinkronisasi (selalu tersedia).

`getCurrentTimestamp(buffer, length)` — ambil waktu saat ini:

- **NTP mode:** jika belum ready → return `"0000-00-00 00:00:00"`. Format: `YYYY-MM-DD HH:MM:SS`.
- **RTC mode:** jika belum ready → return `"0000-00-00 00:00:00"`. Format: `YYYY-MM-DD HH:MM:SS`.

`updateTnow()` — update timestamp di HMI tiap 1 detik:

1. Cek sudah lewat 1 detik sejak update terakhir
2. Ambil timestamp dari `getCurrentTimestamp()`
3. Kirim ke `pageSys.tNow.txt`

Firmware HANYA mengupdate `pageSys.tNow.txt`. Jam di dashboard (`tHH/tMM/tSS`) dikendalikan oleh HMI sendiri via timer internal.

scanner.ino

Input dari barcode scanner via Serial2.

Variabel:

- `scannerBuffer[32]` — buffer sementara untuk menyimpan karakter yang masuk
- `scannerIdx` — indeks karakter terakhir di buffer

`processScannerInput(input)` — proses data dari scanner berdasarkan halaman aktif:

<code>currentPageId</code>	Fungsi dipanggil
<code>PAGE_MLD_F_ID</code>	<code>fillFrontMouldInput(input)</code>
<code>PAGE_MLD_B_ID</code>	<code>fillBackMouldInput(input)</code>
<code>PAGE_LOT_F_ID</code>	<code>fillFrontLotInput(input)</code>
<code>PAGE_LOT_B_ID</code>	<code>fillBackLotInput(input)</code>
<code>PAGE_LOGIN_F_ID</code>	<code>nIdF.setValue(id)</code> — konversi string ke angka
<code>PAGE_LOGIN_B_ID</code>	<code>nIdB.setValue(id)</code> — konversi string ke angka

`handleScanner()` – baca karakter dari Serial2 satu per satu:

1. Jika karakter `\n` atau `\r` → proses data yang terkumpul, reset buffer
 2. Jika bukan newline → tambahkan ke buffer (maks 32 karakter)
-

simulation.ino

Mode simulasi otomatis untuk testing tanpa mesin asli. Juga berisi display helpers untuk update angka di layar.

Variabel:

- `autoFrontEnabled` / `autoBackEnabled` – apakah auto-cycle aktif untuk tiap layer
- `autoFrontIntervalMs` / `autoBackIntervalMs` – interval auto-cycle dalam milidetik
- `autoFrontLastCycleAt` / `autoBackLastCycleAt` – timestamp siklus terakhir

Display helpers:

`updateFrontSimulationDisplay(cycle, output, quota, isi, ng, ok)` – update 6 angka di halaman simulasi: `nSCycF`, `nSOutF`, `nSQuotaF`, `nSIsiF`, `nSNGF`, `nSOKF`.

`updateFrontDashboardDisplay(cycle, output, isi, ok)` – update 4 angka di dashboard: `nFCycD`, `nFOutD`, `nFOutD2` (OK), `nFIsiD`.

`updateBackSimulationDisplay()` / `updateBackDashboardDisplay()` – logika sama untuk layer belakang.

Auto-cycle:

`bAutoEnFCallback()` – tombol enable auto-cycle depan ditekan:

1. Baca interval dari `nAutoMsF` (milidetik)
2. Jika interval 0 → disable, tampilkan "AUTO F MS 0"
3. Jika interval valid → catat waktu mulai, enable auto-cycle

`bAutoDisFCallback()` – disable auto-cycle depan.

`bAutoEnBCallback()` / `bAutoDisBCallback()` – logika sama untuk belakang.

`runAutoFrontCycle()` – jalankan satu siklus otomatis untuk depan:

1. Baca semua nilai dari HMI: ready, siklus, output, kuota, isi, NG
2. Jika gagal baca → disable auto, tampilkan "AUTO F READ NG"
3. Jika `ready==0` → disable, "AUTO F MESIN LOCK"

4. Jika `isi==0` → disable, "AUTO F ISI 0"
5. Jika `quota < isi` → disable, "AUTO F KUOTA KURANG"
6. Jika semua OK → increment siklus, output, kurangi kuota, update semua display

`runAutoBackCycle()` – logika sama untuk belakang.

`updateSimulation()` – dipanggil dari `loop()`. Cek timer untuk kedua layer. Jika waktunya → jalankan auto-cycle.

`registerSimulationCallbacks()` – daftarkan callback tombol auto enable/disable.

Topik MQTT

Topik	Arah	Disimpan	Isi
<code>start/IOTHP-BP/...</code>	Simulator → Firmware	Tidak	"START"
<code>finish/IOTHP-BP/...</code>	Simulator → Firmware	Tidak	JSON siklus
<code>dataA/IOTHP-BP/...</code>	Firmware → Server	Ya	Data layer depan
<code>dataB/IOTHP-BP/...</code>	Firmware → Server	Ya	Data layer belakang
<code>event/IOTHP-BP/...</code>	Firmware → Server	Tidak	Event downtime/NG
<code>CONTROL/IOTHP-BP/...</code>	Server → Firmware	—	"REBOOT"

GPIO (Pin ESP32)

Pin	Fungsi	Arah
GPIO25	Pengaman (interlock)	OUTPUT (HIGH=siap, LOW=lock)
GPIO16	Scanner barcode RX	INPUT
GPIO17	Scanner barcode TX	OUTPUT

Troubleshooting

Masalah	Solusi
Interlock merah	Cek login, mould, lot, isi, kuota
Siklus tidak bertambah	Pastikan interlock hijau, $isi > 0$, $kuota \geq isi$

Masalah	Solusi
Waktu salah	Pastikan WiFi dan internet aktif
Data tidak muncul di server	Cek koneksi WiFi, restart jika perlu
Barcode tidak terbaca	Cek kabel Serial2, baud rate 9600